

EECS 498 · AASE · FALL 2026

Skills and the Elephant-Goldfish Model

LECTURE 10 · OCT 1, 2026

TODAY

Two parts

1. Skills
2. The Elephant-Goldfish Model

Reading: Rensin, "Elephants, Goldfish, and the New Golden Age of Software Engineering"

Assemble, answer, apply



Assemble

Every word the model sees



Answer

Text out, nothing more



Apply

The harness acts on it

Today lives in the first card.

Skills

METHOD THE AGENT LOADS WHEN IT NEEDS IT

DEFINITION

What a skill is

A folder an agent loads when a task needs it.

- **Instructions** - SKILL.md, always
- **Resources** - files it may read
- **Scripts** - code it may run

Written for a model to read, not for a program to call.

A skill on disk

```
elephant/
├── SKILL.md                required
├── references/
│   └── design-doc-sections.md    read on demand
├── assets/
│   ├── design-doc-template.md    copied into output
│   └── elephant-ledger-template.md
└── evals/
    └── trigger-evals.json        tests the description
```

The instructor's elephant skill. No scripts/ here. Lives in `.claude/skills/` or `.agents/skills/`.

The frontmatter

```
name: elephant
description: Runs a no-code design conversation for a new
  feature, refactor, or system change, then writes the
  four-section design doc that becomes the source of truth.
  Use when the user says "design a feature", "let's plan X",
  "spec this out", or otherwise opens a non-trivial design
  problem, even if they did not ask for a document.
  Hand the finished doc to /goldfish. Not for one-line
  fixes or questions about how existing code works.
```

The instructor's elephant skill, trimmed.

The body of SKILL.md

```
# Elephant: grow the design before you grow the code
## EGM operating reflexes (shared preamble)
## Hard rules: do not break these
## Workflow
### Step 2: The "No Code" rule (the interview)
### Step 3: The sycophant defense
### Step 5: Build the design doc, section by section
    Before drafting each one, read its entry in
    references/design-doc-sections.md
## Anti-patterns
## Why this skill exists
```

Headings only, plus the line that sends the model to references/.

The **description** decides **when**.

The body decides how.

Four homes for instructions

Home	Enters the request
System prompt, AGENTS.md	Every request
Tool definition	Every request, as a schema
Skill	When the task calls for it
Your own head	When you retype it

NOT THE SAME THING

Skill versus tool

	Tool	Skill
What it is	A function	Instructions
Who acts	The harness runs it	The model follows it
In the request	Full schema, always	Description, then body
What comes back	A result	A way of working

A tool does something. A skill changes how the model decides what to do.

The cost of knowing everything

What the agent holds	Characters	About
Five descriptions	3,659	900 tokens
Five full SKILL.md files	70,697	18,000 tokens

At roughly four characters a token. Twenty skills is four times this.

One SKILL.md, many harnesses

- **Claude Code** - where the format started
- **Codex CLI and others** - read the same folder
- **Aider** - no skills, yet

Spec: agentskills.io. Each tool documents where it looks for the folder.

The Elephant-Goldfish Model

DESIGN BEFORE CODE

Rensin's article in three parts

Part	Claim
1	A tool, not a toy
2	The Elephant-Goldfish Model
3	We are all managers now

Dave Rensin, Distinguished Engineer, Google. dresin.medium.com

Design is the new code.

The Elephant

- **No code** until the doc passes
- **Interview** before proposing
- **Model drafts first**, you push back
- **One doc**, four sections

The four-section design doc

Section	Holds
Problem	Three to five plain sentences
Technical plan	A block diagram, light on jargon
Alternatives	Rejected ideas, and why
Detailed implementation	Every file, every change

The Assemble question

Twenty skills, each SKILL.md about 4,000 tokens.

1. On turn 1, where does each skill go in the request?
2. What happens before a full SKILL.md appears?
3. Which turn's Assemble does it land in?

On paper, five minutes. Sketch the request.

Back to Assemble

WHERE SKILLS GO

Break answer key

Question	Answer
Turn 1: where does each skill go?	Name and description, system prompt
What comes before the body?	The model calls a load tool
Which turn's Assemble?	The next one, as a tool result

All twenty bodies in the system prompt: about 80,000 tokens, every request.

Three levels of loading

Level	Enters Assemble	Five EGM skills
Name, description	Every request	~900 tokens
SKILL.md body	After the model asks	~3,500 each
references, scripts	When the body points	What it reads

Anthropic's term: progressive disclosure.

One skill across two requests

Request n

system: rules + **skill index**

tools: read_file, load_skill, ...

user: "let's plan a priority flag"

Answer: call load_skill("elephant")

Request n+1

system: unchanged

tools: unchanged

user, then the call, then
tool: elephant's SKILL.md body

Answer: a question for the user

The Goldfish

- **Fresh session**, no memory
- **Only the doc** as input
- **Can it build this?** If not, the doc fails

Three Goldfish reviewers



A: Comprehension

Explain it back



B: Critic

Find the gaps



C: Readiness

Could I build this?

Run in parallel. Their reports drive the next round of edits.

The EGM skill set

Skill	Step	Writes
peanuts	Before: feed the Elephant	A README per directory
elephant	Grow the design	The design doc
goldfish	Test the doc	GOLDFISH.md
egm-implement	Build to the doc	IMPLEMENT.md
mean-review	Tear it apart	MEAN-REVIEW.md

This course's design, six rounds

What	Count
Goldfish rounds	6
Reviewer reports	18
Contradiction caught, round 5	Withheld taskr specs

The pivot prompt: situation

Course Pivot

A feedback session from students has us revamping the course. This is an experimental course so that is alright.

Feedback is in instance/f26/feedback

We are still interested in many of our original design ideas.

Multifaceted pivot is upcoming

- Today's lecture is about Skills and EGM
- Break question gets at how and where skills get included during the Assemble step
- Students were given original Dave Rensin article about EGM last night to read before class today, some of them will have done that
- My implementation of EGM will be discussed and eventually given to them
 - Review the current EGM skill set and update with any best practices for skills that are newer or not implemented already
 - Add a second pass to fully document the EGM skills in a fashion that is meant for instruction so students can read, review, and learn

The pivot prompt: asks

- Current plan for P2 is to have students use a stronger model to adapt Aider to become more like Claude Code. This is the Analyze phase but with a new source repo
- The old repo stops at end of P1, Pair Programmer
- Decisions that need advice
 - Which model should students be using for maximum learning opportunity
 - Which harness "
- An additional 2-3 weeks of roadmap after this pivot needs to be discussed

Also, include this prompt verbatim in this or a future lecture, as an example of design and good prompts

What it does well

Settled and open, kept apart. Evidence named. Asks for advice, not answers.

What a Goldfish flags

No acceptance criteria. "Which harness" unfinished.
No deadline stated.

Rensin, part 1: criteria before output.

CLOSE

Takeaways

- A **skill** is portable context plus a trigger
- The **index** is always in; the body is earned
- A **Goldfish** is a nearly empty Assemble

CLOSE

Before Tuesday

- **Rensin's article**, if not yet read
- **One SKILL.md**, read closely
- **Your description** for one method you repeat

Questions?